

University of Asia Pacific

DEPARTMENT OF CSE

COURSE CODE : 402
COURSE TITLE : Operating System Lab

DATE OF SUBMISSION :17/10/2026

SUBMITTED BY:

NAME : Fairoj Salim
STUDENT ID : 23101165
SECTION : C2
LEVEL : 4th Year 1st Semester

SUBMITTED TO:

Atia Rahman Orthi
LECTURER
Department of CSE, UAP

PROBLEM STATEMENT

Problem 1: FCFS Disk Scheduling

Implement FCFS (First Come First Serve) Disk Scheduling for the following request sequence with the initial head position at 50, and observe the effect of the order of requests on the total head movement:

Case	Request Sequence
Unordered	0, 14, 30, 41, 50, 62, 79, 100, 114, 62
Ordered	0, 14, 30, 41, 50, 62, 62, 79, 100, 114

Calculate the total head movement and show the service sequence for both cases.

Problem 2: SSTF Disk Scheduling

Implement SSTF (Shortest Seek Time First) Disk Scheduling for the request sequence {3, 4, 10, 25, 30, 60} with the initial head position at 41. Calculate the total head movement and show the service sequence, and verify whether the order in which the requests are listed affects the result.

SOURCE CODE

FCFS Disk Scheduling — Unordered Request Sequence

```
req_seq = [0,14,30,41,50,62,79,100,114,62]
head = 50
total_movement = 0
print("FCFS Disk Scheduling")

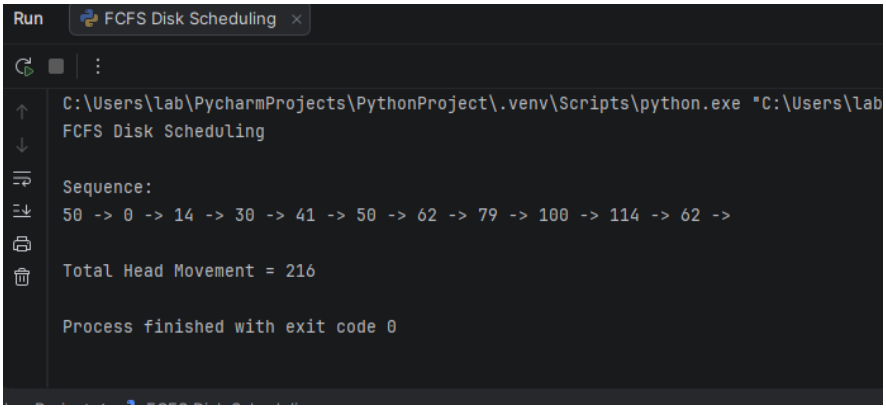
print("\nSequence:")
print(head, end=" -> ")

for i in range(len(req_seq)):

    movement = abs(req_seq[i] - head)
    total_movement = total_movement + movement
    head = req_seq[i]
    print(head, end=" -> ")

print("\n\nTotal Head Movement =", total_movement)
```

OUTPUT



FCFS Disk Scheduling — Ordered Request Sequence

```
req_seq = [0,14,30,41,50,62,62,79,100,114]
head = 50
total_movement = 0
print("FCFS Disk Scheduling")

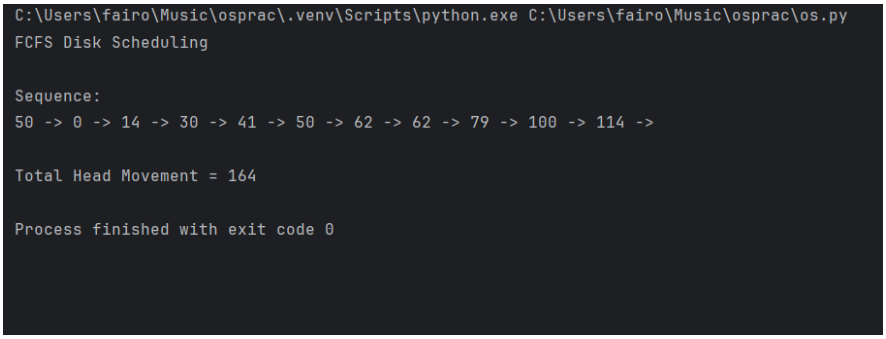
print("\nSequence:")
print(head, end=" -> ")

for i in range(len(req_seq)):

    movement = abs(req_seq[i] - head)
    total_movement = total_movement + movement
    head = req_seq[i]
    print(head, end=" -> ")

print("\n\nTotal Head Movement =", total_movement)
```

OUTPUT



SSTF Disk Scheduling

```
req_seq = [3,4,10,25,30,60]
head = 41
total_movement = 0
visited = [0] * len(req_seq)
sequence = []

while len(req_seq) > 0:
   shortest = 0
   for i in range(1,len(req_seq)):
      if abs(req_seq[i] - head) < abs(req_seq[shortest] - head):
         shortest = i
   movement = abs(req_seq[shortest] - head)
   total_movement = total_movement + movement
   head = req_seq[shortest]

   sequence.append(head)
   req_seq.pop(shortest)

print("SSTF Disk Scheduling")
print("\nSequence:")
print(41, end=" -> ")
for i in range(len(sequence)):
   print(sequence[i], end=" -> ")


print("\n\nTotal Head Movement =", total_movement)
```

OUTPUT



DISCUSSION

In this experiment, FCFS (First Come First Serve) and SSTF (Shortest Seek Time First) Disk Scheduling algorithms were implemented in Python, and the total head movement was calculated and compared using Completion sequence and Total Head Movement as the performance metric.

FCFS: Why the Unordered and Ordered Cases Gave Different Results

- **How FCFS works:** FCFS serves disk requests strictly in the order they appear in the request queue, without rearranging them by distance from the head.
- **Unordered case:** The request 62 appears twice, with the second 62 placed at the very end of the queue, after 114. The head has to travel all the way out to 114 and then come all the way back to 62, giving a Total Head Movement of 216.
- **Ordered case:** The duplicate 62 is placed right next to the first 62, so the head moves smoothly outward (50 → 62 → 62 → 79 → 100 → 114) without any backtracking, giving a lower Total Head Movement of 164.
- **Conclusion:** Since FCFS has no logic to reorder requests, the arrangement of the input queue directly decides how much the head has to travel — a poorly ordered queue causes extra, avoidable seek movement.

SSTF: Why the Order of Requests Did Not Matter

- **How SSTF works:** At every step, SSTF scans all remaining requests and picks the one closest to the current head position, regardless of where that request sits in the input list.
- **Result:** Running the same request set {3, 4, 10, 25, 30, 60} with the list given in a different order still produced the same service sequence (41 → 30 → 25 → 10 → 4 → 3 → 60) and the same Total Head Movement of 95.
- **Why:** SSTF rebuilds its own execution order each time based on distance from the head, not on the order the requests were listed in, so shuffling the input list has no effect on the final schedule or movement.

FCFS vs SSTF: Comparison

- **Total head movement:** SSTF (95) is far lower than either FCFS case (216 unordered, 164 ordered) for comparable request patterns, since SSTF always chooses the nearest request next.
- **Sensitivity to input order:** FCFS is directly affected by the order of the request queue; SSTF is completely unaffected by it.
- **Fairness:** FCFS is fair — every request is served strictly in arrival order with no risk of starvation; SSTF can starve far-away requests if closer requests keep appearing.
- **Simplicity vs efficiency:** FCFS is simpler to implement and predictable; SSTF requires extra computation at every step to find the nearest request but gives noticeably better (lower) head movement.

CONCLUSION

This experiment provided practical experience in implementing FCFS and SSTF Disk Scheduling algorithms using Python and comparing their behavior under different request arrangements.

- **FCFS:** Total head movement changes with the order of the request queue — 216 for the unordered case versus 164 for the ordered case — since FCFS always serves requests strictly in the order given.
- **SSTF:** Total head movement stayed the same (95) regardless of the input order, since SSTF always selects the nearest request first based on distance, not list position.
- **Overall:** SSTF gives better (lower) head movement than FCFS and is immune to how the request queue is arranged, making it more efficient; FCFS remains simpler and fairer but is sensitive to request order and can cause unnecessary extra seek movement.